```
encoderCount = encoderCount ;
}
}
```

In this setup, we tested various values for the proportional gain, Kp. The following figures illustrate the motor's response at different Kp values:

- Figure 4: With Kp = 10, the motor did not move.

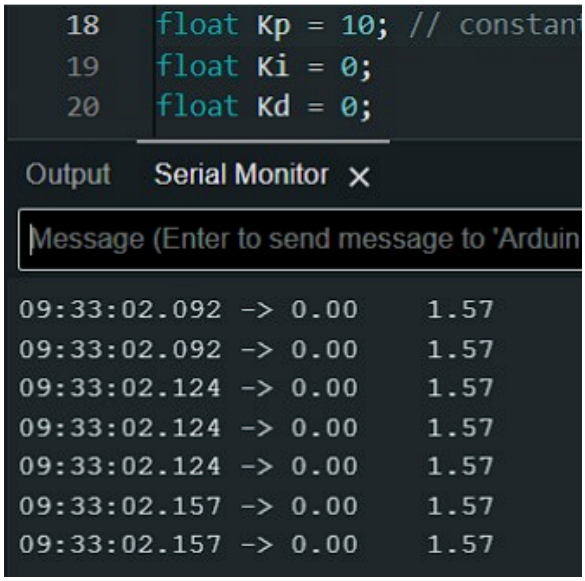


Figure 4: Kp = 10.

- Figure 5: With Kp = 20, the motor moved but significantly undershot the target position.

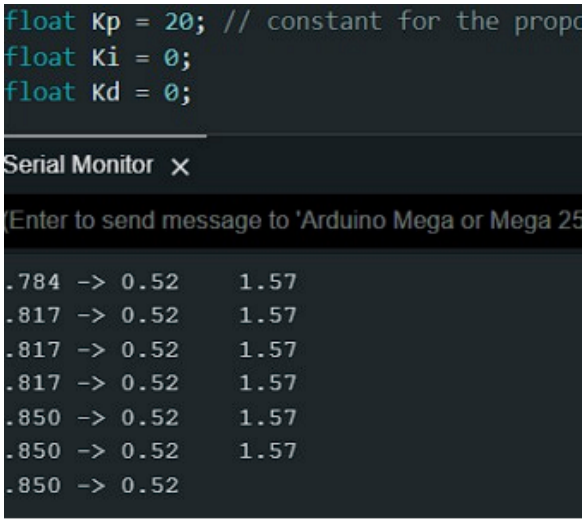


Figure 5: Kp = 20.

- Figure 6: With Kp = 40, the motor's response was closer to the target but involved overshooting.

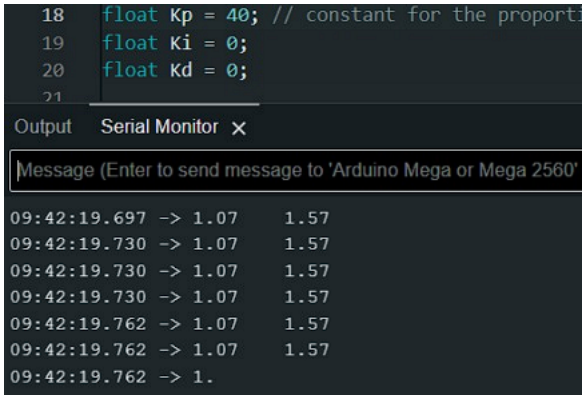


Figure 6: Kp = 40.

2.2.2 Control with the PID

In this part, we modified the Arduino code to incorporate both integral and derivative terms, enhancing the PID control capabilities. These modifications allows for a more complex control strategy, aiming to stabilize the system response. Unfortunatelly, serial plotter was not working properly so we were not able to test the modification and observe the changes.

We have added that part of the code as modification.

```
PreErr = Err ;
Err = Reference - pos ;
//
//
// Write your errors for integral and derivative parts !!!
DErr = ( Err - PreErr )/ interval ;

IErr = (( Err + PreErr )* interval )/2 + PreIErr ;

PreIErr = IErr ;
//
//

// PID control ( edit Ki and Kd to make PI control or PID control )
U = Kp * Err + Kd * DErr + Ki * IErr ; // ADD INTEGRAL AND DERIVATIVE PARTS HERE !!!
```

3 Conclusion

The experiments conducted provided insights into PID control and its practical application to DC motor position management. Adjustments in PID parameters significantly affected the motor's response.